IoT SENTINEL AI

Machine Learning-Based Intrusion Detection for MQTT-Based IoT Networks

TECHNICAL PROJECT REPORT

Prepared by
Hadiya Khan

Abstract

The Internet of Things (IoT) relies on lightweight messaging protocols to connect constrained devices, gateways, and applications. MQTT is widely used in such environments, but its publish/subscribe communication model can also be abused to generate disruptive traffic. IoT Sentinel AI is an experimental machine-learning project that investigates network-traffic-based detection of MQTT message flooding and evaluates a separate multiclass classification task using the MQTTEEB-D dataset.

The project combines a locally controlled MQTT testbed, simulated sensor devices, Mosquitto, packet capture, PCAP feature extraction, model evaluation, an offline inference engine, and a Streamlit dashboard. In the controlled testbed, nine independent runs produced 2,385 telemetry records and 126 PCAP-derived windows. Twelve mixed boundary windows were excluded, leaving 114 windows (102 attack-free and 12 attack-present) for the primary binary evaluation. Random Forest models achieved 100% across the reported primary grouped cross-validation metrics; however, the small number of positive windows means these results must be interpreted cautiously.

A separate MQTTEEB-D evaluation used 222,567 records after excluding 246 records with missing labels. Four-fold Leave-One-Loop-Out evaluation produced a mean accuracy of 49.97% and macro F1 of 54.56%. The final binary inference engine uses four inter-arrival-time features and supports offline CSV inference. The Streamlit dashboard presents project overview, traffic analysis, intrusion detection, model evaluation, and external benchmark results. The system is an offline research prototype and does not perform live network monitoring or prevention.

Keywords

MQTT, Internet of Things, IoT security, intrusion detection, machine learning, Random Forest, traffic analysis, MQTTEEB-D, PCAP, Streamlit.

Executive Summary

IoT Sentinel AI was developed as a portfolio-oriented cybersecurity research project. Its purpose is to explore whether network-traffic characteristics can help distinguish benign MQTT activity from message-flooding activity, and to examine multiclass MQTT traffic classification on an external benchmark.

- Built a local MQTT testbed using Mosquitto and three simulated IoT devices.
- Captured MQTT traffic and extracted window-based features from packet captures.
- Evaluated binary models with grouped cross-validation by independent run.
- Tested held-out message-flooding rates of 2, 5, and 10 messages per second.
- Conducted a separate six-class MQTTEEB-D benchmark using four-fold Leave-One-Loop-Out validation.
- Implemented an offline inference engine and an interactive five-section Streamlit dashboard.

The project demonstrates an end-to-end experimental workflow, while retaining important limitations: a small number of positive windows in the controlled testbed, moderate external multiclass performance, and no live detection capability.

1. Introduction

1.1 Background

IoT deployments connect sensors, actuators, gateways, and cloud services. Messaging protocols are essential for transferring telemetry and control information between these components. MQTT is a lightweight publish/subscribe protocol in which clients communicate through a broker. Its low overhead makes it useful for constrained and intermittently connected environments.

The same characteristics that make MQTT practical also create security considerations. A broker or its clients may be affected by excessive message rates, unauthorized publishing, malformed traffic, or other forms of abuse. Traffic-based monitoring can provide signals about abnormal communication patterns, including changes in packet timing and message frequency.

1.2 Problem Statement

MQTT-based IoT networks require practical methods for identifying suspicious traffic without assuming that every deployment has extensive host-level telemetry. This project investigates the use of network traffic features and machine learning for identifying MQTT message-flooding activity in a controlled environment, and separately studies multiclass MQTT traffic classification using an external dataset.

1.3 Aim and Objectives

- Construct a repeatable local MQTT testbed with simulated IoT telemetry.
- Capture and process MQTT network traffic into labeled analysis windows.
- Evaluate supervised and baseline approaches for binary attack detection.
- Measure performance when individual flooding rates are held out from training.
- Evaluate a separate multiclass task on MQTTEEB-D using capture-loop separation.
- Build an offline prediction interface and dashboard for result inspection.

1.4 Scope

The controlled experiment focuses on MQTT message flooding only. It does not represent a comprehensive test of MQTT vulnerabilities or all IoT attack classes. The external MQTTEEB-D benchmark is a separate multiclass experiment and is not merged with the controlled testbed results.

2. Technical Background

2.1 MQTT Communication

MQTT uses a broker-mediated publish/subscribe model. Publishers send messages to topics; subscribers receive messages for topics to which they are subscribed. The broker routes messages between clients. This architecture reduces direct coupling between devices but makes broker availability and message-handling capacity important operational concerns.

2.2 MQTT Message Flooding

Message flooding refers to generating a high volume of messages over a short period. In a controlled experiment, changing the configured publishing rate provides a way to observe how traffic timing and packet-level features vary with load. In this project, message flooding is the only attack family implemented in the local testbed.

2.3 Machine Learning for Traffic Classification

Supervised classifiers learn patterns from labeled examples. Random Forest combines multiple decision trees and can model nonlinear relationships between input features. Isolation Forest is an anomaly-detection approach that isolates observations through random feature and split selection. A dummy classifier provides a simple reference baseline. Performance must be interpreted in light of class balance, sample size, and the independence of training and evaluation data.

2.4 Packet Capture and Feature Extraction

Packet capture records network frames for offline inspection. TShark, the command-line component of Wireshark, can decode packet protocols and export selected fields. The project uses packet-capture-derived windows to construct network features. Labels are annotations for evaluation and are not used as model predictors.

3. System Architecture and Design

The project has two distinct experimental branches that feed into a common reporting and visualization layer.

Component	Purpose
Simulated IoT devices	Publish sensor telemetry such as temperature, humidity, battery level, and status.
Mosquitto broker	Receive and route MQTT messages in the local testbed.
Traffic capture	Record network packets for offline analysis.
Feature extraction	Decode relevant MQTT traffic and aggregate packet observations into windows.
Controlled-testbed branch	Perform binary attack-free versus attack-present classification.
External benchmark branch	Perform six-class classification on MQTTEEB-D.
Offline inference engine	Load the final binary model and classify feature rows or CSV data.
Streamlit dashboard	Present experiment analysis, predictions, model evaluation, and benchmark results.

3.1 Data Flow

Simulated devices publish telemetry to the local broker. Traffic is captured and stored in packet-capture files. The extraction pipeline derives features and associates windows with experiment labels. The controlled testbed dataset is used for binary evaluation, while MQTTEEB-D is processed and evaluated independently. Saved evaluation outputs are displayed in the dashboard. The inference engine accepts prepared feature data and performs offline predictions.

3.2 Separation of Experimental Branches

The two branches should not be interpreted as one combined training dataset. They have different class definitions, feature sets, data sources, and validation strategies. The binary testbed results describe a narrowly controlled flooding experiment. MQTTEEB-D results describe a separate six-class benchmark across capture loops.

4. Experimental Environment and Data Collection

4.1 Environment

Item	Configuration
Operating system	Windows 11
Programming language	Python 3.13
MQTT broker	Mosquitto 2.1.2
MQTT client library	Paho MQTT 2.2.0
Packet analysis	Wireshark / TShark
Dashboard	Streamlit 1.67.0
ML and data processing	Scikit-learn, Pandas, NumPy
Testing	Pytest

4.2 Simulated Devices and Telemetry

Three simulated devices publish JSON telemetry to MQTT topics. The payloads include temperature, humidity, battery level, and device status. This provides repeatable application-level traffic for the controlled testbed.

4.3 Controlled Flooding Rates

The message-flooding scenario was configured at 2, 5, and 10 messages per second. The rates were used in held-out evaluation so that one configured rate could be excluded from training while the other two were used for model development.

4.4 Experiment Summary

Measure	Count
Independent runs	9
Telemetry records	2,385
PCAP-derived windows	126
Mixed boundary windows excluded	12
Primary evaluation windows	114
Attack-free windows	102
Attack-present windows	12

The 12 mixed boundary windows were excluded from the primary binary evaluation to avoid using windows containing transitions between phases as clean single-class examples. The resulting primary dataset is imbalanced, with 12 positive windows out of 114.

5. Data Processing and Feature Engineering

5.1 PCAP Processing

The extraction workflow uses TShark to decode MQTT traffic from packet captures. Relevant MQTT PUBLISH traffic is filtered and packet observations are aggregated into fixed-duration windows. The extraction process records timing, packet size, stream, port, and MQTT-related information where available.

5.2 Window-Level Features

The project explored packet counts, MQTT message counts, byte totals, packet sizes, packet rates, publish rates, inter-arrival timing, TCP stream counts, and source-port counts during dataset investigation and model development.

5.3 Final Inference Features

Feature	Description
mean_interarrival_seconds	Mean time between observed packets/messages in the window.
std_interarrival_seconds	Standard deviation of inter-arrival times.
min_interarrival_seconds	Minimum observed inter-arrival time.
max_interarrival_seconds	Maximum observed inter-arrival time.

The final offline binary inference model uses these four inter-arrival features. The label is used as the evaluation target, not as a predictor. Feature values must be computed consistently with the training pipeline.

5.4 Data Quality and Labeling

Window labels are derived from decoded MQTT payloads and experiment metadata in the controlled testbed. Label columns are retained for evaluation and auditing. Mixed boundary windows are treated separately from the primary evaluation set. Data cleaning and quality checks are important because missing values, infinite values, duplicates, and ambiguous phase boundaries can affect evaluation.

6. Machine Learning Methodology

6.1 Binary Classification Task

The controlled-testbed task is binary classification with two outcomes: attack_free and attack_present. The aim is to identify windows associated with the configured MQTT message-flooding phase.

6.2 Models

- Random Forest (Network): a supervised tree-ensemble model evaluated using network-oriented features.
- Random Forest (Combined): a supervised model using the combined feature representation evaluated in the experiment.
- Isolation Forest (Network): an unsupervised anomaly-detection baseline.
- Dummy Baseline: a simple reference classifier used to contextualize model performance.

6.3 Validation Strategy

The primary binary evaluation uses three-fold StratifiedGroupKFold with grouping by independent run_id. Grouping is intended to keep windows from the same experimental run together rather than distributing them across training and validation folds. This reduces the risk of overly optimistic evaluation caused by related windows from the same run appearing on both sides.

6.4 Evaluation Metrics

Accuracy measures the proportion of correct predictions. Precision measures the proportion of predicted positives that are correct. Recall measures the proportion of actual positives detected. F1 is the harmonic mean of precision and recall. Balanced accuracy averages recall across classes and is useful when class frequencies differ. False-positive rate measures the proportion of negative examples incorrectly flagged. ROC-AUC and average precision summarize ranking performance across thresholds.

7. Controlled Testbed Results

7.1 Primary Binary Evaluation

Model	Accuracy	Balanced Acc.	Precision	Recall	F1
Dummy Baseline	89.47%	50.00%	0.00%	0.00%	0.00%
Random Forest (Network)	100.00%	100.00%	100.00%	100.00%	100.00%
Random Forest (Combined)	100.00%	100.00%	100.00%	100.00%	100.00%
Isolation Forest (Network)	61.40%	78.43%	21.43%	100.00%	35.29%

For both Random Forest models, the reported confusion-matrix counts were TN=102, FP=0, FN=0, and TP=12. The Isolation Forest result was TN=58, FP=44, FN=0, and TP=12. Its recall was 100%, but the false-positive rate was 43.14%, indicating that many attack-free windows were also flagged.

The Random Forest results are perfect on this particular primary evaluation. They should not be generalized beyond the tested data. There are only 12 attack-present windows, so the positive-class evidence is limited. Additional independent runs and broader attack scenarios are required for stronger conclusions.

7.2 Held-Out Flooding-Rate Evaluation

Held-out rate	Training rates	Test windows	Accuracy
2 msg/s	5 and 10 msg/s	8	100%
5 msg/s	2 and 10 msg/s	8	100%
10 msg/s	2 and 5 msg/s	8	100%

This experiment evaluates different configured intensities of the same message-flooding attack family. It is not an unseen-attack-family evaluation. The test set contains eight windows per held-out rate, so the results are promising within the controlled setup but remain limited in scale.

7.3 Interpretation

The high Random Forest scores indicate that the selected features separated the labeled windows in the current controlled dataset. The baseline results show why accuracy alone can be misleading in an imbalanced dataset: the dummy baseline achieved 89.47% accuracy while detecting none of the attack-

present windows. Balanced accuracy, recall, precision, F1, and confusion-matrix counts therefore provide important additional context.

8. External MQTTEEB-D Benchmark

8.1 Dataset Summary

Dataset measure	Value
Original records	222,813
Rows excluded due to missing labels	246
Usable records	222,567
Classes	6

Class	Records
Legitimate	17,557
DoS	41,802
Flood	10,213
Brute Force	27,377
Malformed	105,528
SlowITe	20,090

8.2 Validation Method

The external benchmark uses four-fold Leave-One-Loop-Out validation. In each fold, three capture loops are used for training and one complete loop is held out for testing. This design evaluates performance across capture loops and reduces overlap between training and test capture groups.

8.3 Overall Results

Metric	Score
Mean accuracy	49.97%
Balanced accuracy	62.36%
Macro precision	57.66%
Macro recall	62.36%
Macro F1	54.56%
Weighted F1	51.47%

8.4 Class-Wise F1 Results

Class	F1 score

Legitimate	26.71%
DoS	38.09%
Flood	96.20%
Brute Force	83.66%
Malformed	52.05%
SlowITe	30.66%

Flood achieved the strongest F1 score (96.20%), followed by Brute Force (83.66%). Legitimate traffic had the weakest F1 score (26.71%). This uneven performance indicates that the model distinguishes some attack classes more effectively than others. The overall macro F1 of 54.56% provides a more balanced view than accuracy alone.

The Leave-One-Loop-Out design evaluates generalization across capture loops. It does not establish generalization to unseen attack families. The external benchmark is independent of the controlled testbed binary evaluation and should be reported separately.

9. Offline Inference Engine

The project includes a Python inference engine that loads the final binary Random Forest artifact, validates the expected input feature fields, and predicts a class for a traffic-window feature record. It also calculates class probabilities and reports prediction execution time. A CSV-based batch inference path is available.

9.1 Prediction Output

- Predicted class (attack-free or attack-present).
- Attack-free probability.
- Attack-present probability.
- Prediction execution time in milliseconds.

9.2 Inference Benchmark

The benchmark used 1,000 sequential single-row prediction calls after 100 warm-up calls. Feature validation occurs before the measured prediction section. Model loading and warm-up are excluded.

Metric	Measured value
Mean latency	76.772 ms
Median latency	76.488 ms
Minimum latency	73.9455 ms
Maximum latency	111.3691 ms
95th percentile	79.398 ms
99th percentile	81.304 ms
Standard deviation	2.116 ms
Throughput	13.025 predictions/second

The timing includes model prediction, probability calculation, and Python call overhead after feature validation. It is not an end-to-end MQTT latency measurement and does not establish performance on embedded or resource-constrained devices.

10. Streamlit Dashboard

The Streamlit dashboard provides an interactive interface for exploring the project's saved results and performing offline CSV inference. It contains five main sections.

10.1 Overview

The overview page introduces the project and displays summary information. It also provides a CSV upload workflow for running offline detection and inspecting the latest predictions.

10.2 Traffic Analysis

The traffic analysis page supports experiment selection and visual inspection of packet rates, inter-arrival patterns, traffic distributions, actual-versus-predicted comparisons, and tabular results.

10.3 Intrusion Detection

The intrusion detection page accepts a prepared CSV file, runs the offline Random Forest inference engine, displays predicted classes and probability information, and allows results to be downloaded.

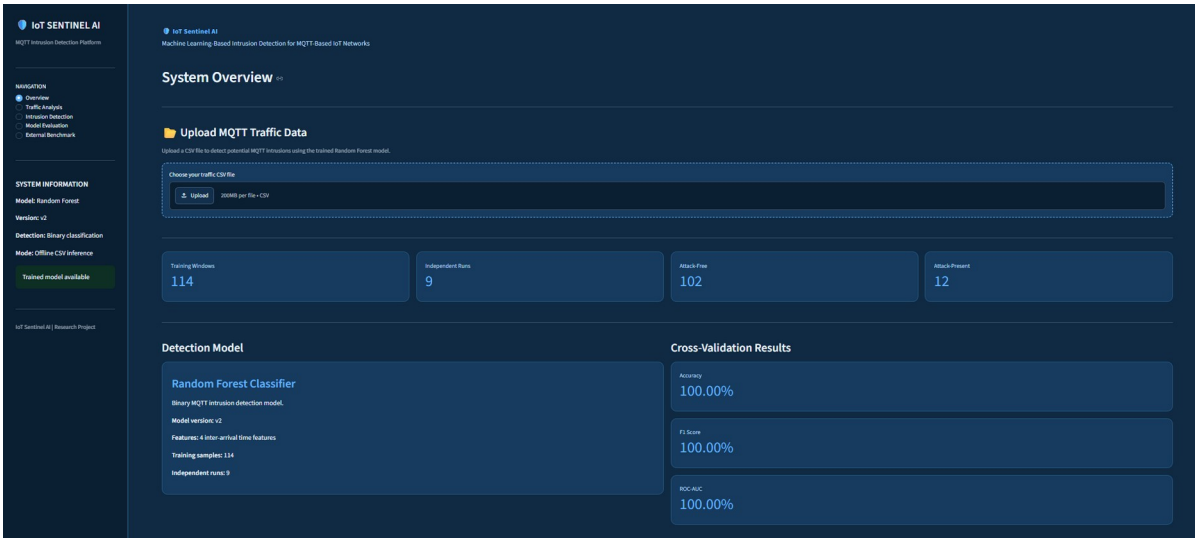
10.4 Model Evaluation

The model evaluation page presents model comparison results, F1 metrics, confusion-matrix counts, and held-out flooding-rate evaluation.

10.5 External Benchmark

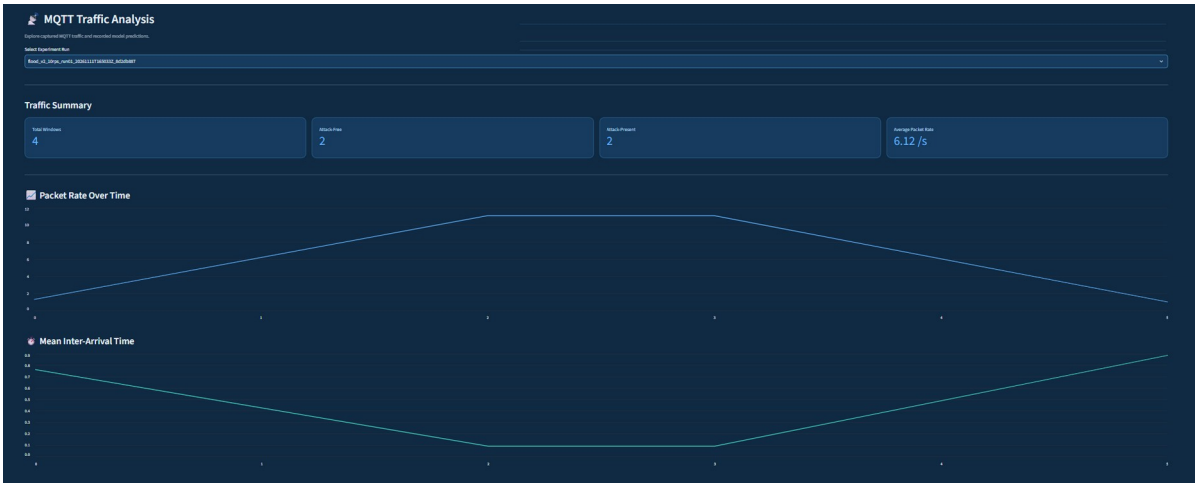
The external benchmark page presents MQTTEEB-D overall metrics, fold-level results, and class-wise performance. It uses saved evaluation reports; the large external benchmark model artifact is not required to display these results.

Dashboard overview



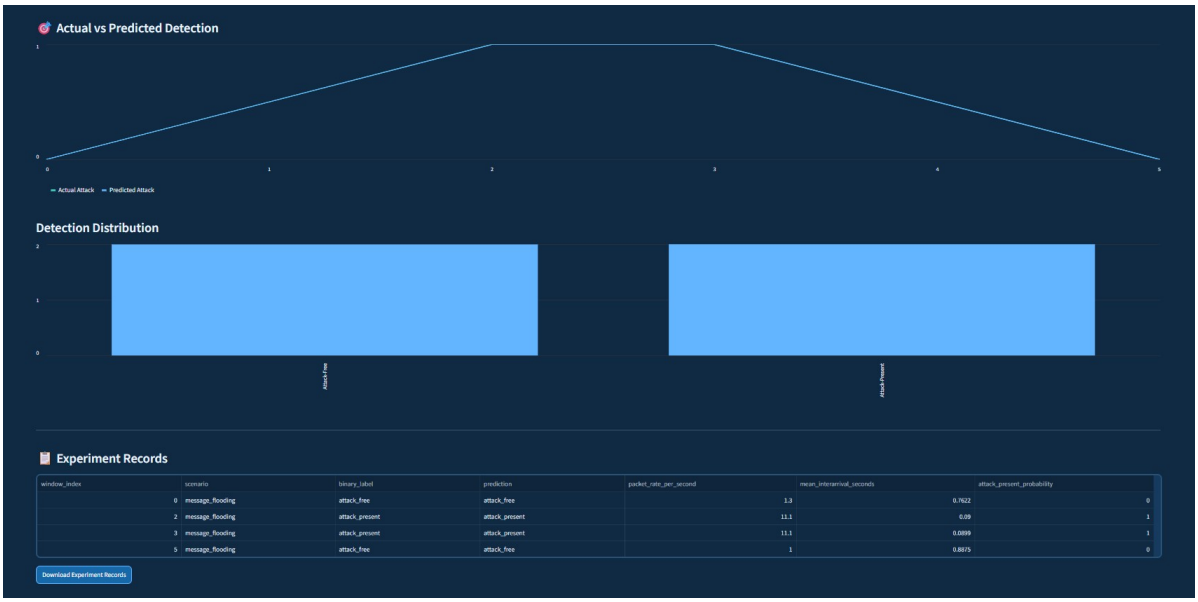
Dashboard overview

Traffic analysis — experiment and traffic overview



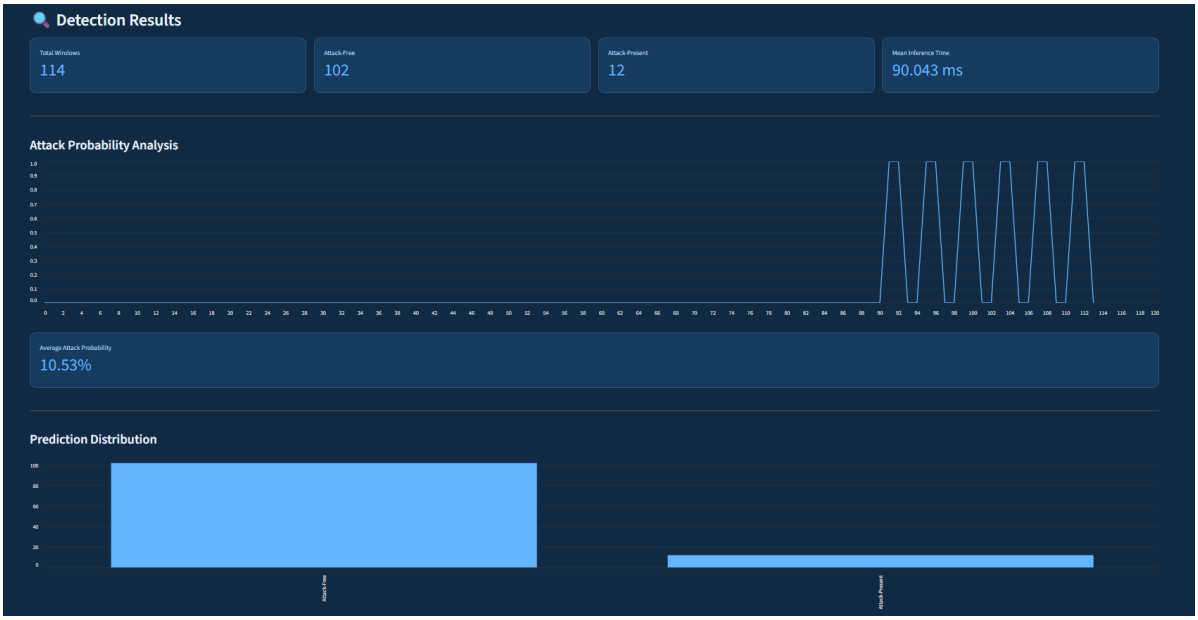
Traffic analysis — experiment and traffic overview

Traffic analysis — detailed visualizations



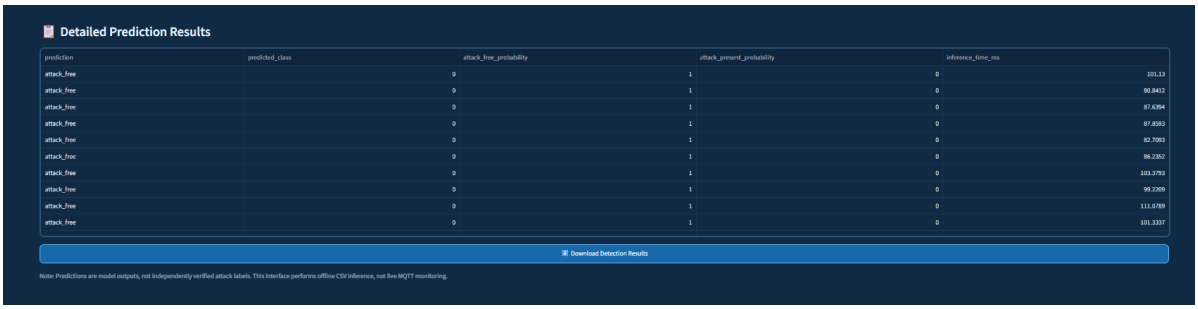
Traffic analysis — detailed visualizations

Intrusion detection — input and predictions



Intrusion detection — input and predictions

Intrusion detection — detailed results



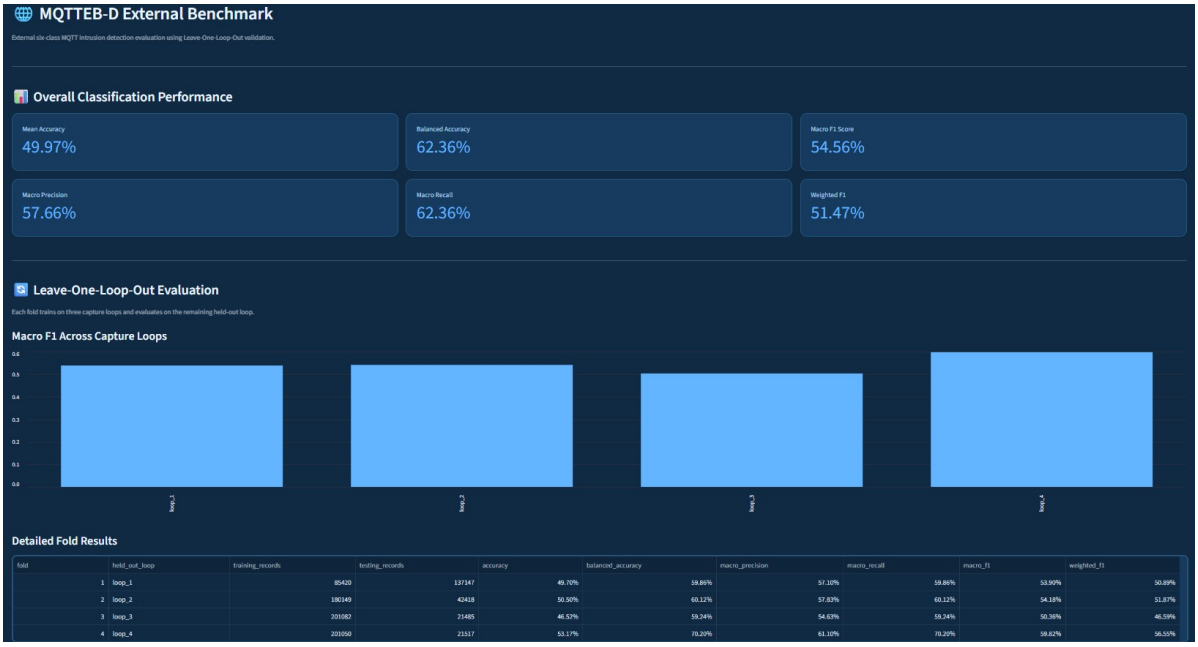
Intrusion detection — detailed results

Model evaluation



Model evaluation

External MQTTEEB-D benchmark — summary



External MQTTEEB-D benchmark — summary

External MQTTEEB-D benchmark — class results



External MQTTEEB-D benchmark — class results

11. Software, Reproducibility and Repository

The public source repository contains the Streamlit dashboard, project scripts, selected model artifacts, selected evaluation reports, dependency files, and this documentation. Raw packet captures and raw datasets are not distributed in the repository. The MQTTEEB-D dataset must be obtained separately for reproducing the external benchmark training workflow. The large trained MQTTEEB-D multiclass model is excluded from GitHub.

Repository: https://github.com/khhadiya-09/IoT-Sentinel-AI

11.1 Running the Dashboard

Create and activate a Python virtual environment, install the dependencies from requirements.txt, and run the dashboard from the project root.

```
python -m venv .venv

.venv\Scripts\Activate.ps1

python -m pip install -r requirements.txt

streamlit run dashboard.py
```

11.2 Running Tests

```
python -m pytest tests/ -v
```

The inference-engine tests are located in tests/. The test command should be run from the repository root after installing the development dependencies.

12. Limitations, Risks and Threats to Validity

- The controlled testbed contains only one attack family: MQTT message flooding.
- The primary binary evaluation contains 12 attack-present windows, which is a small positive sample.
- Perfect primary cross-validation scores do not prove real-world or broad attack generalization.
- Held-out rate evaluation covers rates within the same flooding family, not previously unseen attack families.
- The external MQTTEEB-D benchmark has moderate overall multiclass performance and uneven class-wise results.
- Leave-One-Loop-Out validation measures cross-loop performance, not unseen-family generalization.
- The dashboard performs offline CSV inference and does not capture or classify live MQTT traffic.
- Inference timing was measured in a desktop Python environment, not on an embedded IoT device.
- Reproduction of all experiments requires access to the raw datasets and packet captures, which are not included in the public repository.

These limitations define the appropriate interpretation of the reported results. They also identify the most important areas for future validation.

13. Future Work

- Expand the controlled testbed to include additional MQTT attack scenarios and benign workload diversity.
- Increase the number of independent runs and attack-present windows.
- Evaluate across additional devices, brokers, network conditions, and capture environments.
- Investigate feature and model improvements for legitimate, DoS, and SlowITe classes in MQTTEEB-D.
- Perform external validation on independently collected MQTT traffic.
- Investigate streaming feature extraction and real-time inference as a separate future capability.
- Evaluate resource consumption and latency on representative IoT hardware.
- Add explainability and model-monitoring methods to support security analyst interpretation.

14. Conclusion

IoT Sentinel AI demonstrates an end-to-end experimental workflow for machine-learning-based analysis of MQTT traffic. It combines a controlled IoT testbed, packet capture, feature extraction, binary classification, held-out flooding-rate evaluation, a separate external multiclass benchmark, an offline inference engine, and an interactive dashboard.

The controlled Random Forest models achieved perfect reported metrics on the primary grouped evaluation, but the small number of positive windows requires cautious interpretation. The external MQTTEEB-D benchmark achieved a macro F1 score of 54.56%, with strong results for Flood and Brute Force and weaker performance for Legitimate and SlowITe. Taken together, the findings show both the potential and the limitations of traffic-based MQTT classification in the evaluated settings.

The current implementation is an offline research prototype rather than a live intrusion detection or prevention system. The project provides a documented foundation for further experimentation, independent validation, and future work in IoT and MQTT security.

References and Resources

- MQTT — OASIS Standard. https://mqtt.org/
- Eclipse Mosquitto. https://mosquitto.org/
- Eclipse Paho MQTT Python Client. https://eclipse.dev/paho/
- Wireshark / TShark documentation. https://www.wireshark.org/docs/
- Scikit-learn documentation. https://scikit-learn.org/stable/
- Streamlit documentation. https://docs.streamlit.io/
- Pytest documentation. https://docs.pytest.org/
- MQTTEEB-D dataset: cite the original dataset publication and source documentation used to obtain the dataset in the final academic version.

Note: The original MQTTEEB-D publication citation should be completed from the exact dataset paper/source used in the experiment. It has not been guessed here.